AI ASSISTED CODING-13.3

NAME:P.AJAY

ROLL NO:2403A510B4

BATCH:05

……

TASK 1 – REMOVE REPETITION

PROMPT:

Refactor the following redundant code into a cleaner, modular design using dictionary-based dispatch or separate functions.

CODE:

```
import math
def rectangle_area(x, y):
    return x * y

def square_area(x):
    return x * x

def circle_area(r):
    return math.pi * r * r

area_calculators = {
    "rectangle": lambda x, y: rectangle_area(x, y),
    "square": lambda x, y=0: square_area(x),
    "circle": lambda x, y=0: circle_area(x),
}
def calculate_area(shape, x, y=0):
    if shape not in area_calculators:
        return f"Unknown shape: {shape}"
    return area_calculators[shape](x, y)

# Test
print(calculate_area("rectangle", 5, 3))
print(calculate_area("square", 4))
print(calculate_area("circle", 7))
```

OUTPUT:

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/
Y/ai coding/ai coding/13.3.py"
15
16
153.93804002589985
```

OBSERVATION:

Before: Repeated formulas and long if-elif chain.
After: Modular functions with dictionary-based dispatch; easier to extend with new shapes.

TASK 2 – ERROR HANDLING IN LEGACY CODE

PROMPT:

Refactor this legacy file-reading function with safe error handling and best practices.

CODE:

```python
def read_file(filename):
    """Reads content from a file safely with error handling."""
    try:
        with open(filename, "r") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        return None
    except Exception as e:
        print(f"An error occurred: {e}")
        return None

# Testing
print(read_file("sample.txt"))   # Works if file exists
print(read_file("missing.txt"))  # Triggers error message
```

OUTPUT:

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "
Y/ai coding/ai coding/13.3.py"
Error: File 'sample.txt' not found.
None
Error: File 'missing.txt' not found.
None
```

OBSERVATION:

Before: No error handling; risk of file not closing on error.
After: Uses with open() + try-except; safer, more robust.

TASK 3 – COMPLEX REFACTORING (CLASS DESIGN)

PROMPT:

Refactor this legacy class to improve readability, naming, modularity, and Pythonic style.

CODE:

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks  # list of marks

    def details(self):
        """Displays student details."""
        print(f"Name: {self.name}, Age: {self.age}")

    def total(self):
        """Returns the total marks obtained."""
        return sum(self.marks)

# Testing
s = Student("Alice", 20, [85, 90, 95])
s.details()
print(s.total())
```

OUTPUT:

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/ai coding/palindrome.py"
Name: Alice, Age: 20
270
```

OBSERVATION:

Before: Poor variable names, individual marks stored separately.
After: Clear variable names, marks stored in a list, added docstrings for clarity.

TASK 4 – INEFFICIENT LOOP REFACTORING

PROMPT:

Refactor this inefficient loop into a more Pythonic version.

CODE:

```python
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [i * i for i in nums]

print(squares)
squares = list(map(lambda x: x * x, nums))
print(squares)
```

OUTPUT:

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Lo
Y/ai coding/ai coding/13.3.py"
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
PS C:\Users\AJAY\ai coding\ai coding>
```

OBSERVATION:

Before: Verbose loop with .append().
After: Cleaner, concise list comprehension with identical results.